

CentralImmo

Rapport d'architecture et de conception

Péronnal

Juin 2026

Résumé

CentralImmo est une plateforme d'agrégation immobilière et d'intelligence de marché pour le Cameroun. Le présent rapport illustre l'architecture générale du système au moyen de onze diagrammes réalisés en TikZ : architecture générale, déploiement AWS EC2, modèle de données, framework de scraping, pipeline d'ingestion, orchestration des crawls, résolution des doublons (DCS), scraper universel, pipeline analytique, flux de requêtes API et machine à états du review des annonces. Le backend FastAPI est hébergé sur une instance AWS EC2, avec une base PostgreSQL managée par Amazon RDS, derrière un équilibreur de charge Application Load Balancer et une distribution CloudFront pour le front web.

Table des matières

1	Architecture générale du système	2
2	Déploiement sur AWS EC2	3
3	Modèle de données (ER)	4
4	Framework de scraping	5
5	Pipeline d'ingestion	6
6	Orchestration des crawls (séquence)	7
7	Résolution des doublons (DCS)	8
8	Scraper universel	10
9	Pipeline analytique et scoring	11
10	Flux de requêtes API	12
11	Machine à états du review	13
12	Synthèse et conventions clés	14

Chapitre 1

Architecture générale du système

La plateforme s'organise en quatre plans fonctionnels : la collecte (scrapers Python), la persistance (PostgreSQL multi-couches), l'API (FastAPI) et la présentation (Flutter multi-plateforme). Le scheduler APScheduler s'exécute en arrière-plan au sein du processus FastAPI.

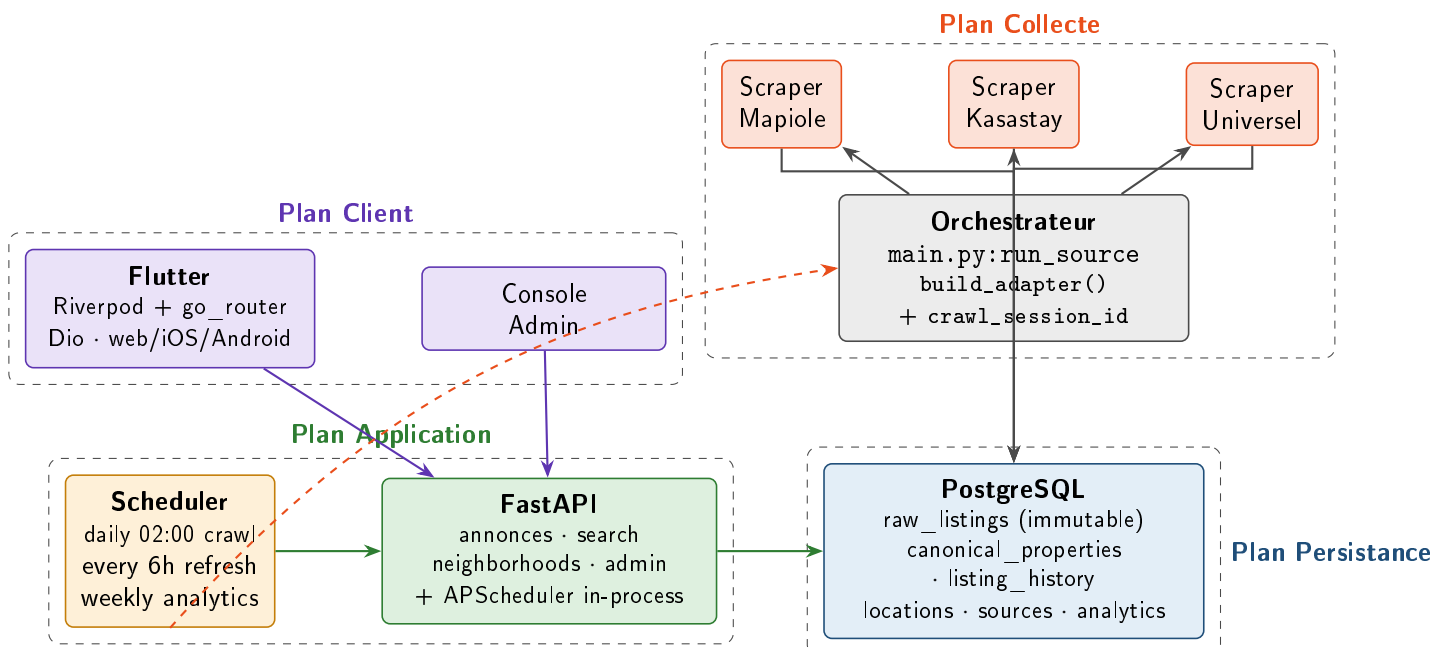


Figure 1.1: Architecture en quatre plans de CentralImmo.

Chapitre 2

Déploiement sur AWS EC2

Le backend FastAPI est hébergé sur une instance EC2 (Amazon Linux 2023, type `t3.medium`) dans un VPC privé. La base PostgreSQL est déléguée à Amazon RDS pour la sauvegarde automatique et le bascuement multi-AZ. La distribution CloudFront sert le build web Flutter et met en cache les images d'annonces. Route 53 résout `centralimmo.cm` vers l'ALB.

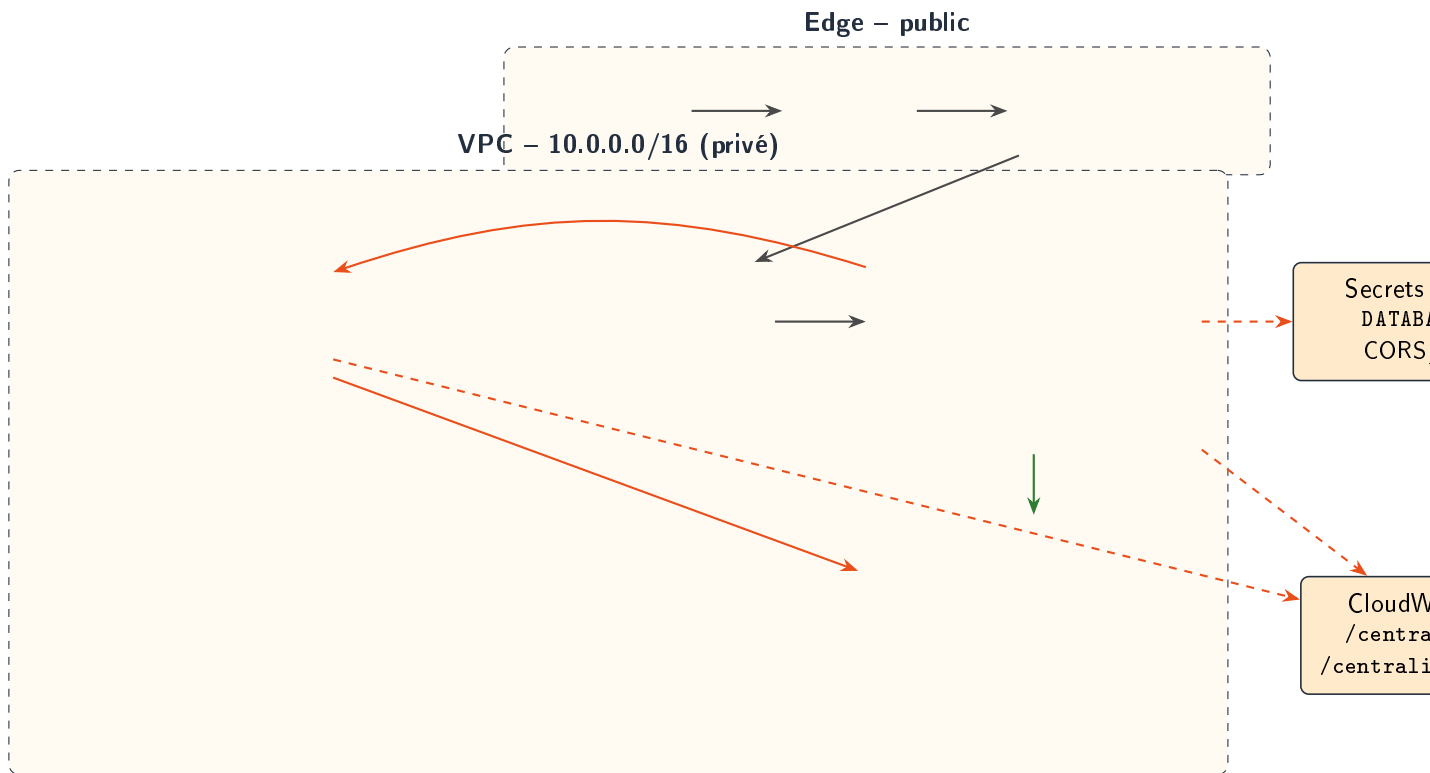


Figure 2.1: Topologie de déploiement sur AWS (EC2 + RDS + CloudFront).

Chapitre 3

Modèle de données (ER)

Huit tables structurent la persistance : deux de référence (**sources**, **locations**), trois cœur (**raw_listings**, **canonical_properties**, **listing_history**) et trois opérationnelles (**image_cache**, **scheduler_jobs**, **neighborhood_analytics**). Les règles d'immuabilité et d'append-only sont matérialisées par les cardinalités.

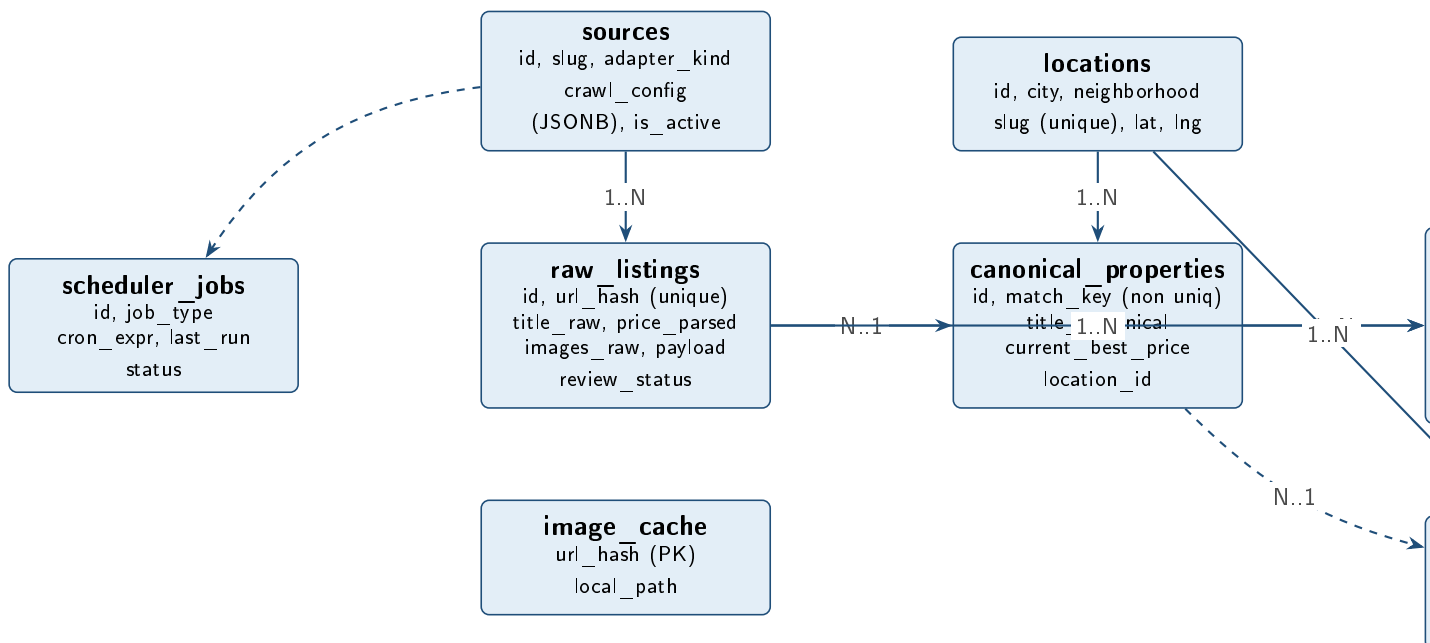


Figure 3.1: Schéma entité-association (PK en orange, FK en bleu).

Chapitre 4

Framework de scraping

Tous les adaptateurs implémentent l'ABC `SourceAdapter` (`fetch_listings`, `fetch_details`, `normalize_data`, `validate_data`) et héritent de `BaseFetchMixin` pour la couche HTTP anti-bot. Le scraper universel s'appuie sur les modules `extractors` et `confidence`.

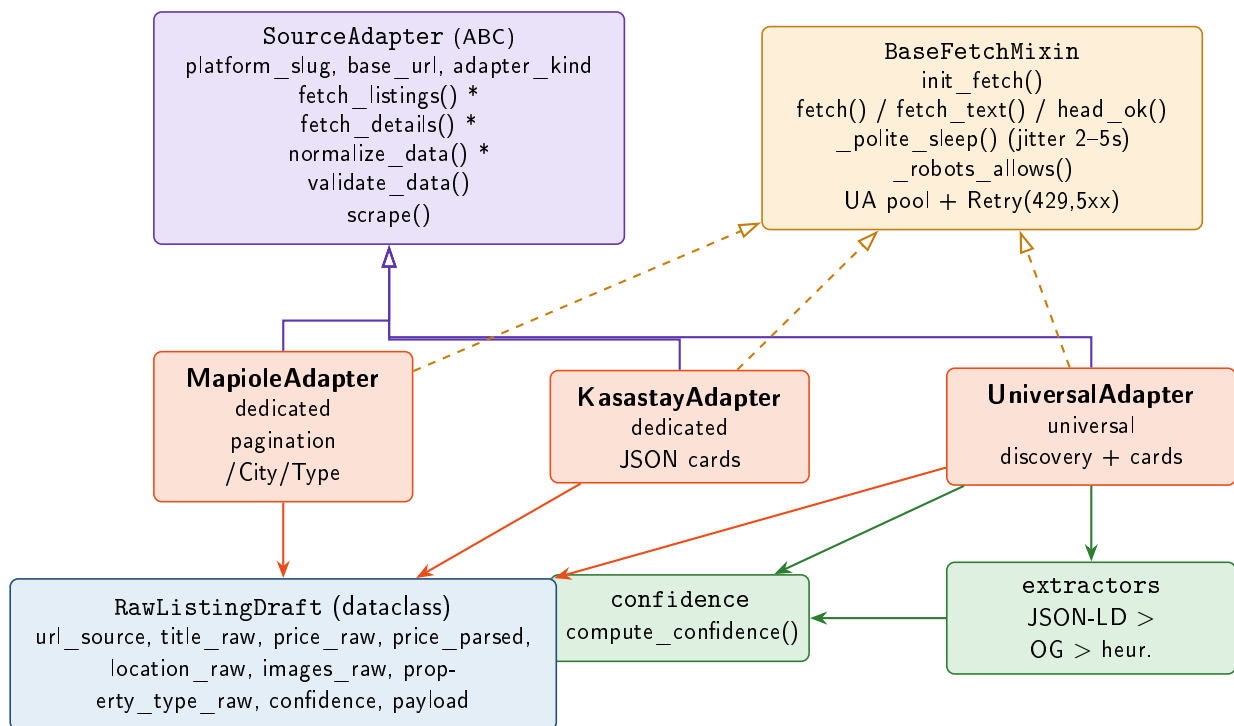


Figure 4.1: Hiérarchie d'adaptateurs et mixin anti-bot.

Chapitre 5

Pipeline d'ingestion

`core/ingest.py` transforme les `RawListingDraft` en lignes `raw_listings` + `canonical_properties` + `listing_history`. L'idempotence est garantie par `url_hash = sha256(url_source)` : un re-crawl ne crée jamais de nouvelle ligne, il compare le prix à la dernière `listing_history` et n'ajoute un événement `price_change` que si la valeur a bougé.

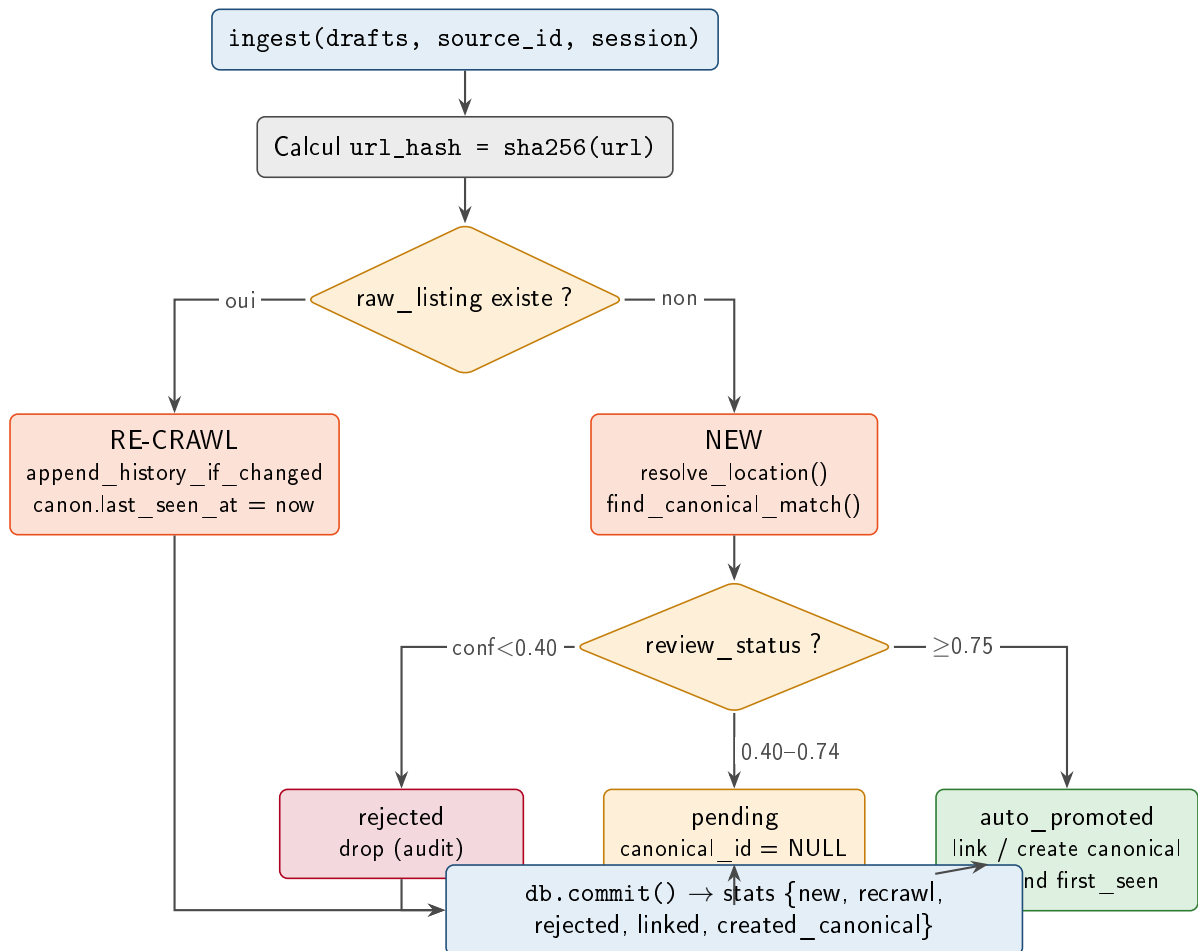


Figure 5.1: Logique d'ingestion idempotente (re-crawl vs nouveau).

Chapitre 6

Orchestration des crawls (séquence)

La séquence ci-dessous décrit l'exécution d'une source par le scheduler (cron quotidien 02:00 UTC) ou par appel manuel `POST /admin/sources/{slug}/crawl`.

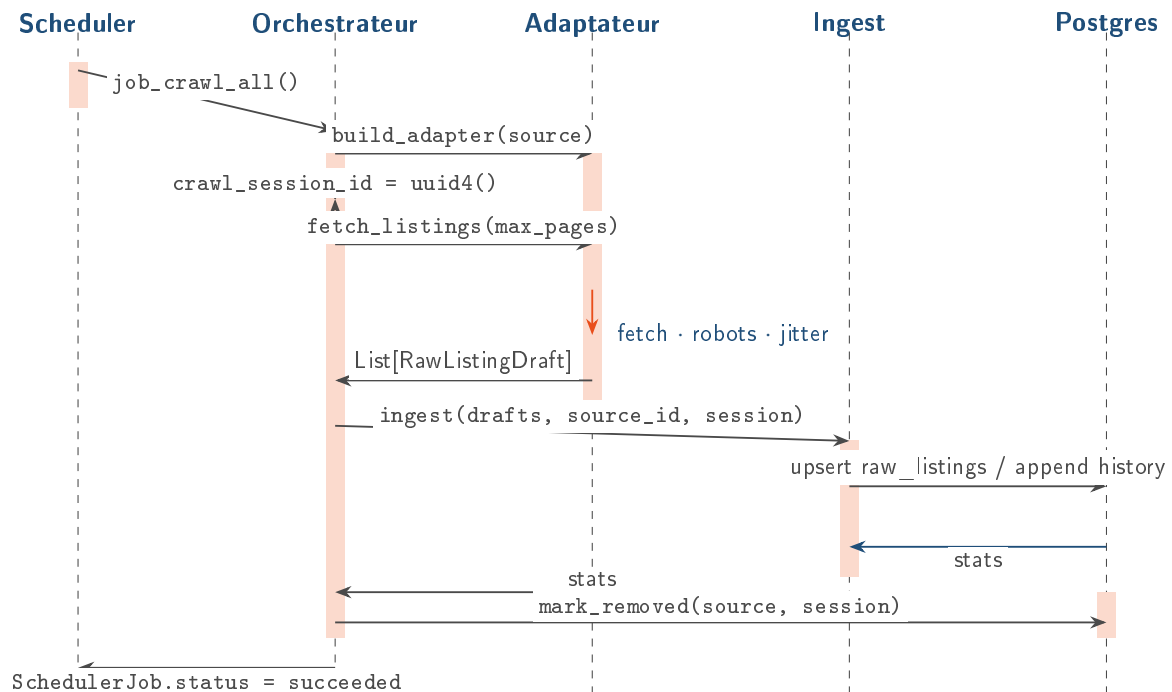


Figure 6.1: Diagramme de séquence d'un crawl source complet.

Chapitre 7

Résolution des doublons (DCS)

Le *Duplicate Confidence Score* (DCS) remplace le scan $\pm 15\%$ sur les prix. Un `match_key` grossier (`sha1(type + city + neighborhood)`) récupère par index un petit ensemble de candidats, puis un score composite pondéré désigne le meilleur et une classification en déduit l'action.

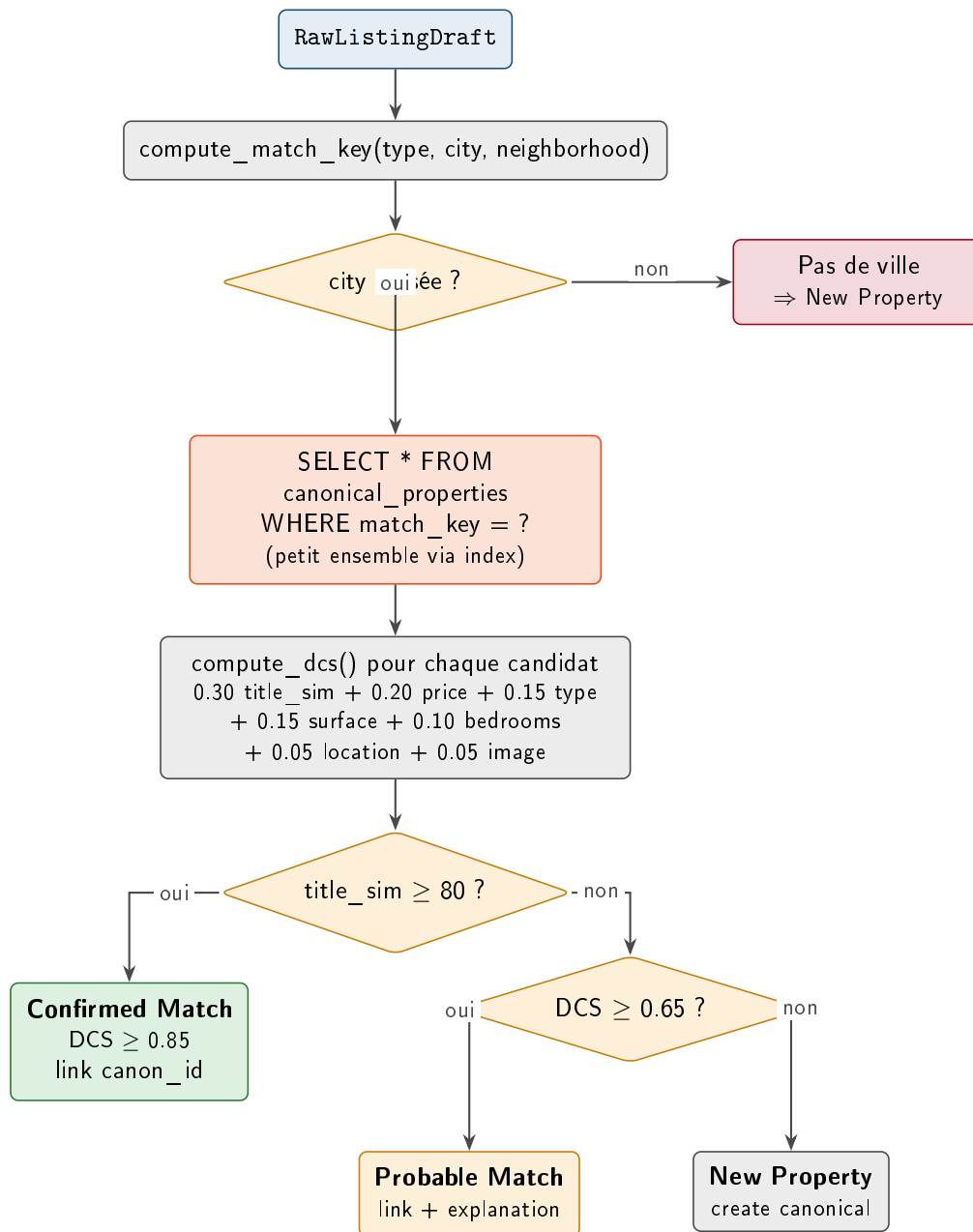


Figure 7.1: Arbre de décision du matcher DCS.

Chapitre 8

Scraper universel

Le scraper universel permet l'ajout d'un nouveau site en quelques minutes, sans adaptateur dédié. Quatre étapes : découverte d'URLs, détection de cartes, extraction multi-source, scoring de confiance.

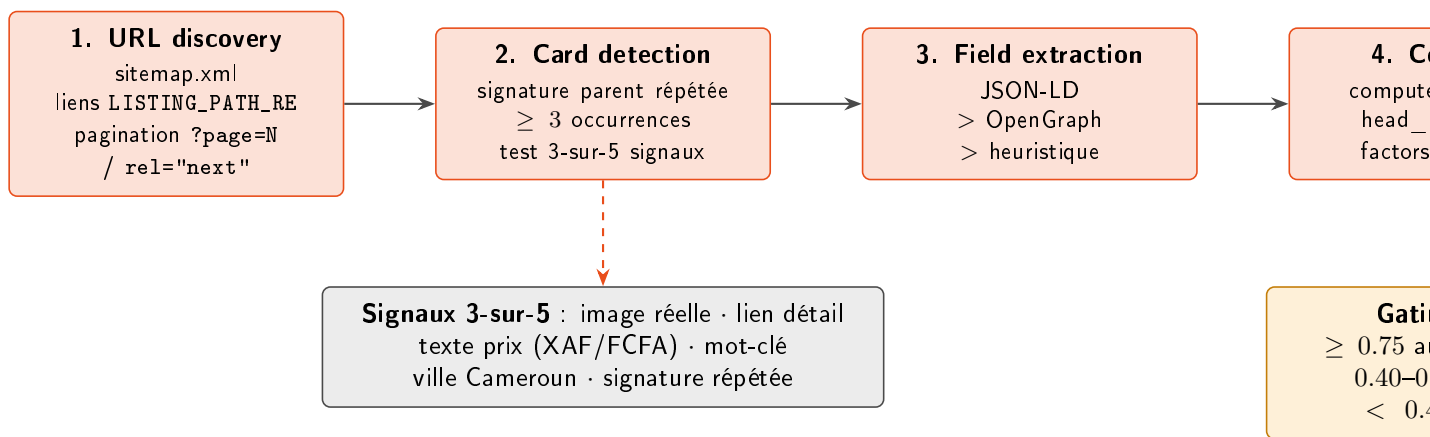


Figure 8.1: Pipeline en quatre étapes du scraper universel.

Chapitre 9

Pipeline analytique et scoring

`core/analytics.py:recompute_all` parcourt toutes les `locations`, calcule pour chacune (et par type de bien) des agrégats de prix, des indicateurs de marché et cinq scores normalisés 0–10 produits par `core/scoring.py`. Le résultat est upserté dans `neighborhood_analytics`, lu ensuite par l'API `GET /neighborhoods/{slug}`.

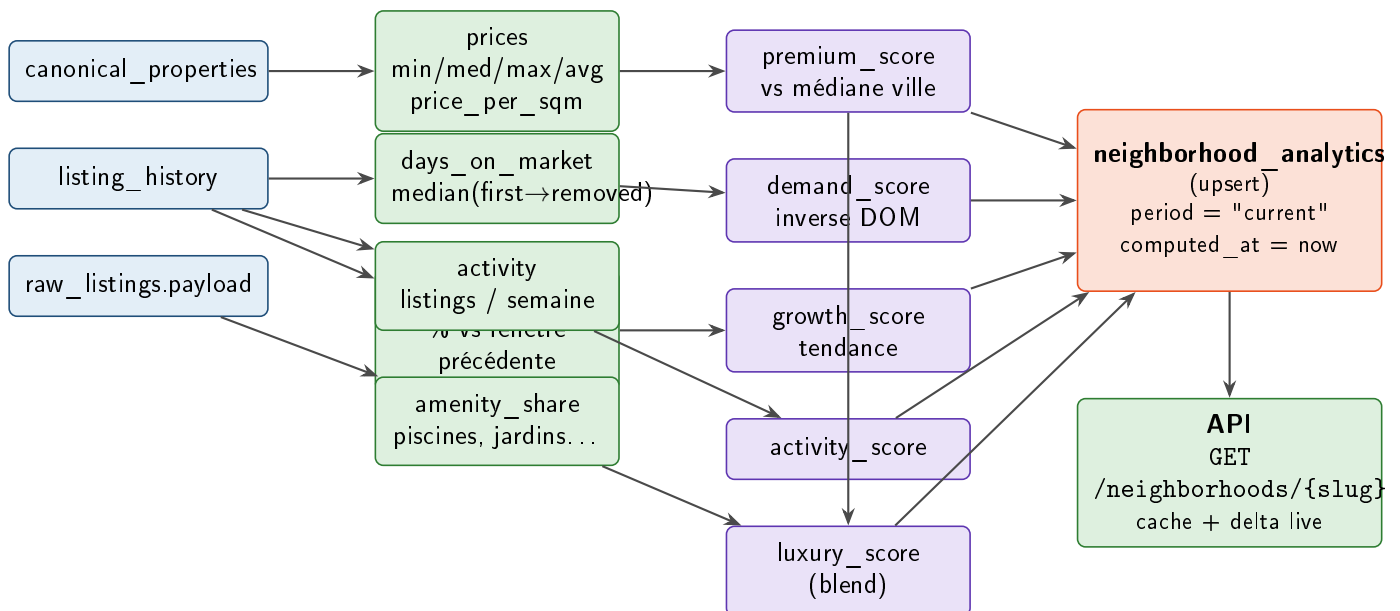


Figure 9.1: Du brut au score : pipeline de `recompute_all`.

Chapitre 10

Flux de requêtes API

Les clients Flutter consomment l'API via Dio (**ApiClient**). Les filtres sont appliqués côté serveur (jamais en client). L'API FastAPI monte cinq routeurs, chacun avec injection de **Session** SQLAlchemy.

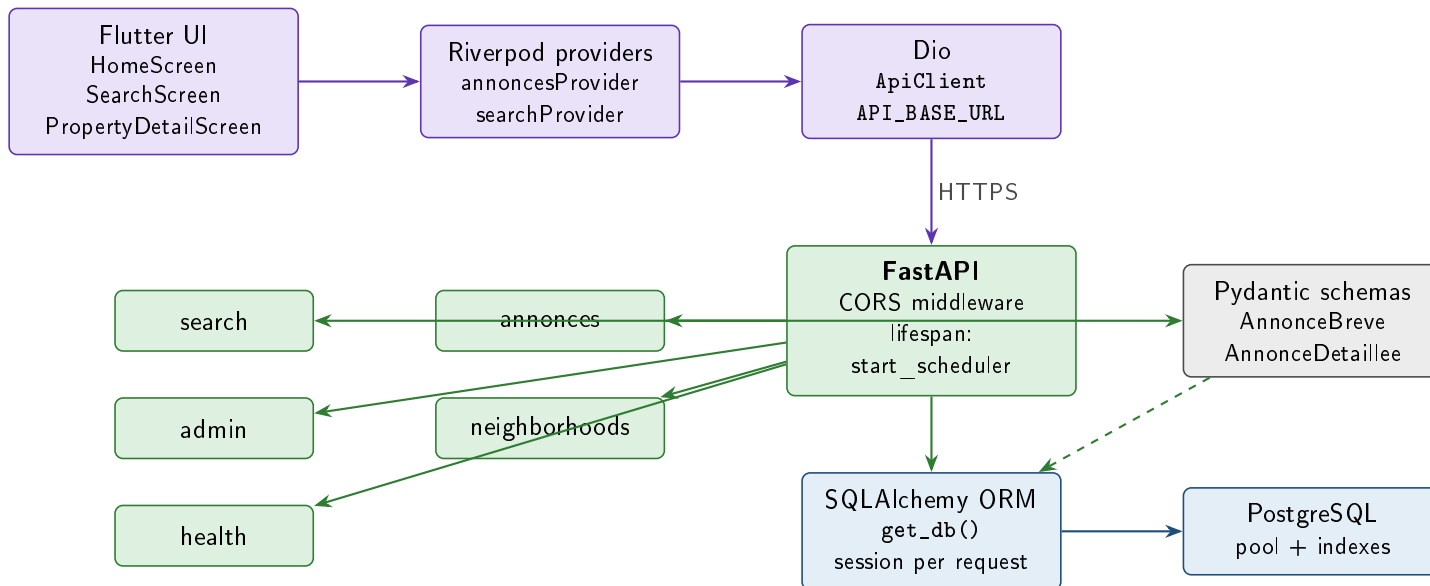


Figure 10.1: Flux d'une requête de l'écran Flutter jusqu'à PostgreSQL.

Chapitre 11

Machine à états du review

Le `review_status` d'un `raw_listing` suit le cycle de vie d'une annonce issue du scraper universel. Les sources dédiées (`dedicated`) entrent directement en `auto_promoted`. Seul l'admin peut décider de la promotion d'une annonce `pending` via `POST /admin/review`.

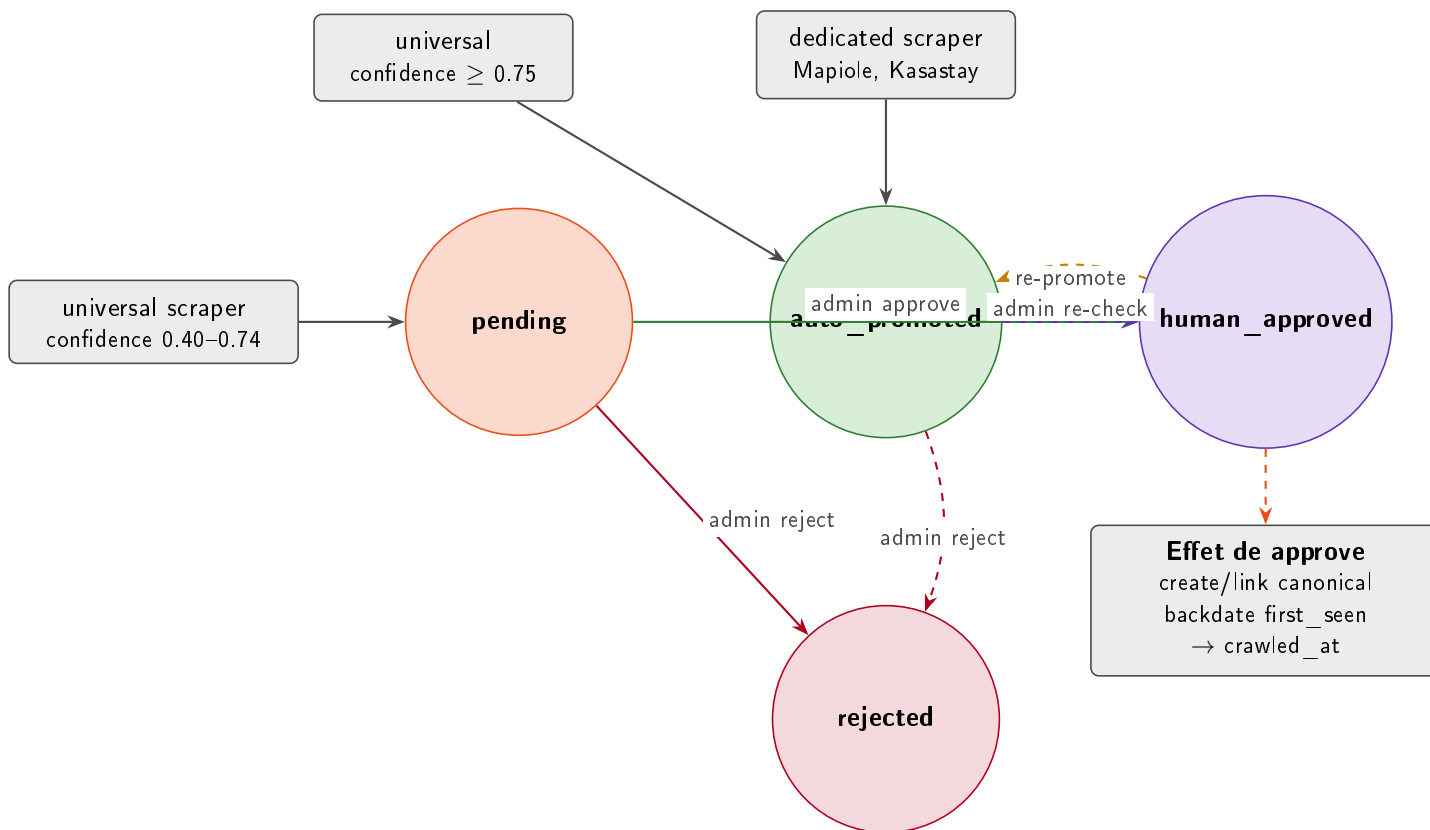


Figure 11.1: Cycle de vie du `review_status` d'un `raw_listing`.

Chapitre 12

Synthèse et conventions clés

- **raw_listings** est *immutable* : un re-crawl ne modifie jamais la ligne, il ajoute un événement dans **listing_history**.
- **listing_history** est *append-only* : aucun UPDATE, aucune suppression.
- **canonical_properties.match_key** est un bucket de récupération de candidats, *non unique* ; le DCS désambiguïse.
- Les **pending** ne créent pas de canonical (évite les orphelins).
- Le **scheduler** vit dans le processus FastAPI (**APScheduler** BackgroundScheduler) et déclenche : crawl quotidien 02:00, refresh toutes les 6h, analytics hebdo lundi 03:00.
- Les scrapers respectent **robots.txt** via **BaseFetchMixin**, avec UA navigateur, jitter 2–5 s, retry exponentiel sur 429/5xx, délai 15 s.
- Les filtres de l'API sont *server-side* ; le client Flutter ne filtre jamais.
- L'état Flutter est géré par Riverpod, le routage par **go_router**, l'HTTP par Dio avec **AppConfig.apiUrl**.
- Déploiement AWS : EC2 **t3.medium** pour l'app, RDS PostgreSQL multi-AZ, CloudFront pour le web et les images, Route 53 pour le DNS, Secrets Manager pour les credentials.